

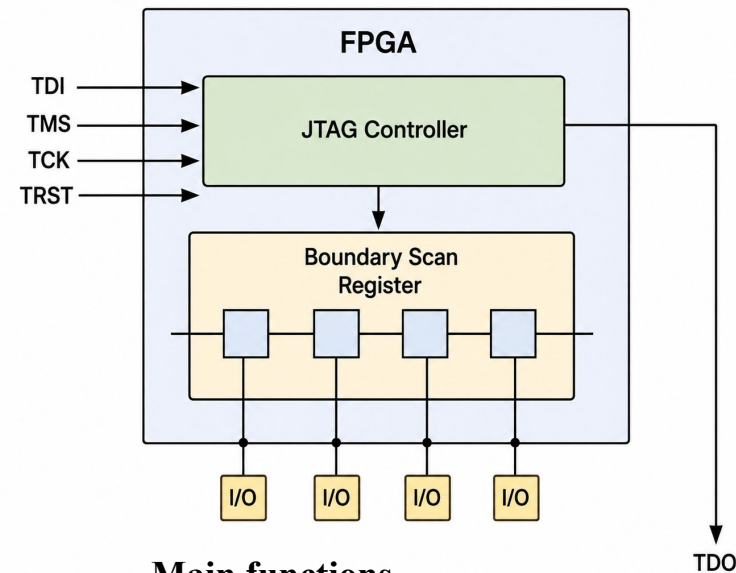
# **Module 6: Programming the FPGA**

# Boundary Scan

- **Boundary Scan** is a testing and debugging technique used to test the connections between an FPGA and other devices on a PCB without requiring direct physical access to every signal pin.
- It is based on the **IEEE 1149.1 JTAG (Joint Test Action Group)** standard.
- **Basic concept:** A boundary-scan cell is placed near the I/O pin of the FPGA. These cells are connected together to form a Boundary Scan Register (BSR).

## Example

Consider an FPGA output is connected to an external memory input. Boundary Scan can test whether the connection between these two devices is functioning correctly.



## Main functions

Boundary Scan can be used for:

- Testing PCB interconnections
- Detecting open connections
- Detecting short circuits
- Testing FPGA I/O connectivity
- In-system programming
- Debugging hardware

| Signal      | Meaning          |
|-------------|------------------|
| <b>TDI</b>  | Test Data In     |
| <b>TDO</b>  | Test Data Out    |
| <b>TMS</b>  | Test Mode Select |
| <b>TCK</b>  | Test Clock       |
| <b>TRST</b> | Test Reset       |

# Boundary Scan

## Main functions

Boundary Scan can be used for:

- Testing PCB interconnections
- Detecting open connections
- Detecting short circuits
- Testing FPGA I/O connectivity
- In-system programming
- Debugging hardware

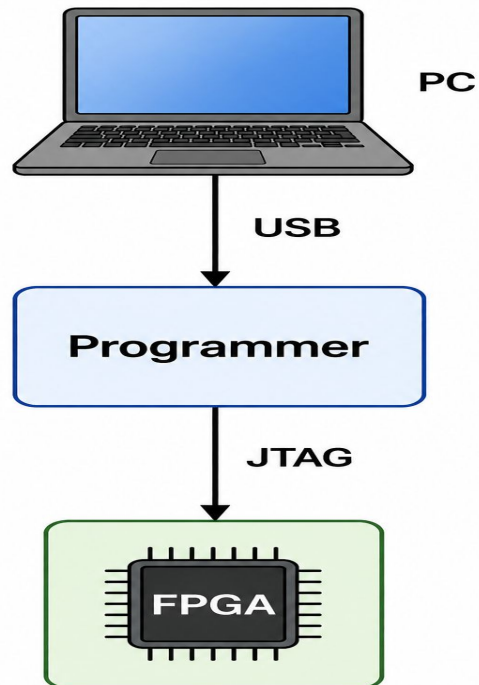
## Advantages

- Reduces need for physical probing
- Useful for densely populated PCBs
- Supports automated testing
- Enables in-system programming
- Useful for hardware debugging

# Programming the FPGA

FPGA programming means configuring the programmable logic resources of an FPGA according to the required digital circuit.

Unlike software programming, FPGA programming generally means creating a configuration bitstream that determines how the FPGA's LUTs, flip-flops, routing resources, I/O blocks, RAMs, and other resources operate.



## Typical Quartus Prime flow

For an Altera/Intel FPGA:

### 1. Create project

Create a new project in Quartus Prime.

### 2. Write HDL

### 3. Compile the design

### 4. Generate programming file:

The FPGA configuration file is generated.

### 5. Program FPGA

The file is downloaded into the FPGA using a programming interface such as JTAG.

### 6. Configuration technologies

Depending on the FPGA family, configuration can involve:

JTAG

Active Serial

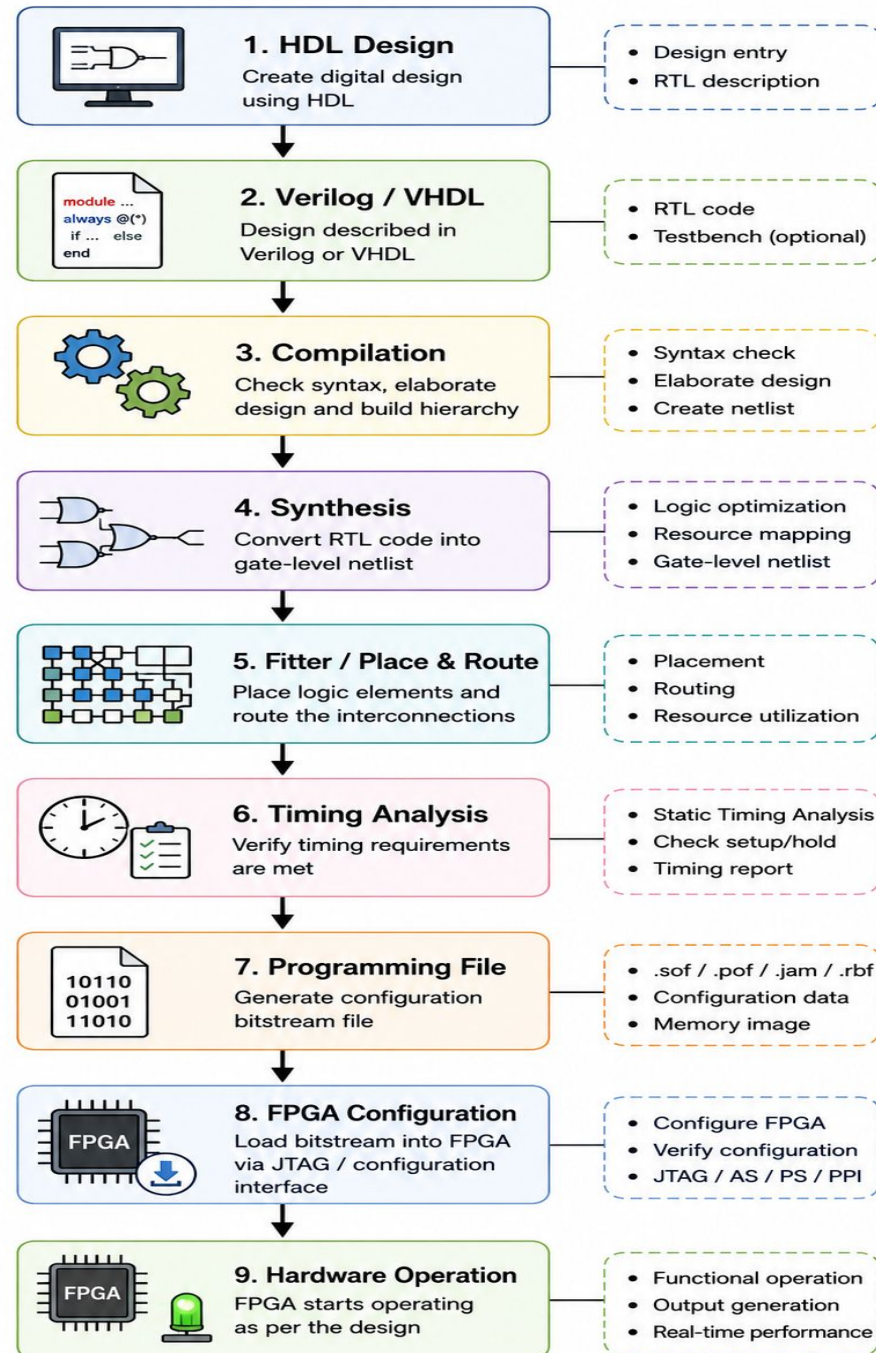
Passive Serial

Passive Parallel

Configuration memory

For development boards, JTAG programming is commonly used.

# FPGA DESIGN FLOW



# Constraint Editor

The Constraint Editor is used to specify design constraints that tell FPGA design software how and where the design should operate.

Constraints are important because the FPGA tools need information about:

- Pin assignments
- Clock frequencies
- Input/output timing
- I/O standards
- Timing requirements
- Physical placement

## Example

Suppose an FPGA has a 50 MHz clock.

The clock period is:

$$T = 1/f$$

Therefore,

$$T = 1/50\text{MHz} = 20\text{ns}$$

The timing constraint tells the FPGA tool that the clock period is **20 ns**.

# Types of constraints

## A. Pin constraints

Specify which physical FPGA pin is connected to a signal.

### Example:

- `clk → PIN_23`
- `reset → PIN_45`
- `LED → PIN_67`

## B. I/O constraints

Specify:

- I/O voltage standard
- Drive strength
- Pull-up/pull-down settings

## Constraint file

Modern FPGA design flows commonly use **SDC (Synopsys Design Constraints)** for timing constraints.

### Example:

```
create_clock -name clk -period 20.0 [get_ports clk]
```

This defines a 50 MHz clock.

## Importance

Without proper constraints, the FPGA tools may not correctly understand the required operating speed or physical connections.

## C. Timing constraints

Specify:

- Clock period
- Input delay
- Output delay
- Setup requirements
- Hold requirements
- False paths
- Multicycle paths

# Static Timing Analysis (STA)

Static Timing Analysis is the process of verifying whether a digital circuit can operate correctly at the required clock frequency without actually applying input test vectors.

STA analyzes the timing paths in the design.

## Important timing parameters

1. Setup time
2. Hold time

## Clock-to-clock timing

A typical path is:

FF1 → Combinational Logic → FF2

STA checks whether:

$$T_{clk} \geq TCQ + T_{logic} + T_{setup}$$

where:

- $T_{clk}$  = clock period
- $TCQ$  = clock-to-Q delay
- $T_{logic}$  = combinational logic delay
- $T_{setup}$  = setup time

## Importance

STA helps determine the maximum safe operating frequency of the FPGA design and identifies timing-critical paths.



# Static Timing Analysis (STA)

**Static Timing Analysis (STA)** is a method used in digital circuit and FPGA/VLSI design to verify whether all timing requirements of a circuit are satisfied **without applying simulation vectors**. It mathematically analyzes the delays of signals through combinational and sequential logic and determines whether the circuit can operate correctly at the specified clock frequency.

STA is especially important in **FPGA design, ASIC design, VLSI, and HDL-based digital system design**.

## Why Static Timing Analysis is Required:

A digital circuit may be logically correct but still fail because signals do not reach their destinations within the required time.

For example, consider:

Suppose:

Clock period = 10 ns

FF1 launches data at the clock edge.

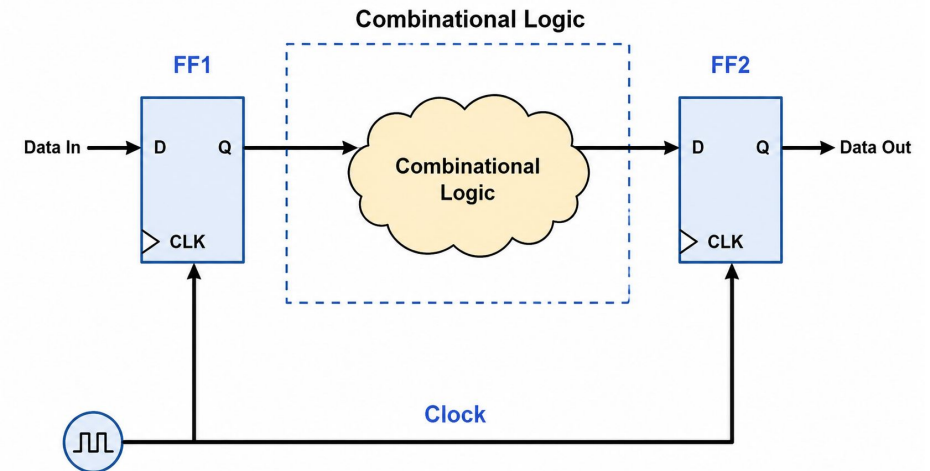
Data passes through combinational logic.

FF2 must capture the data at the next clock edge.

If the data requires **12 ns** to reach FF2, the circuit violates its timing requirement.

Therefore:

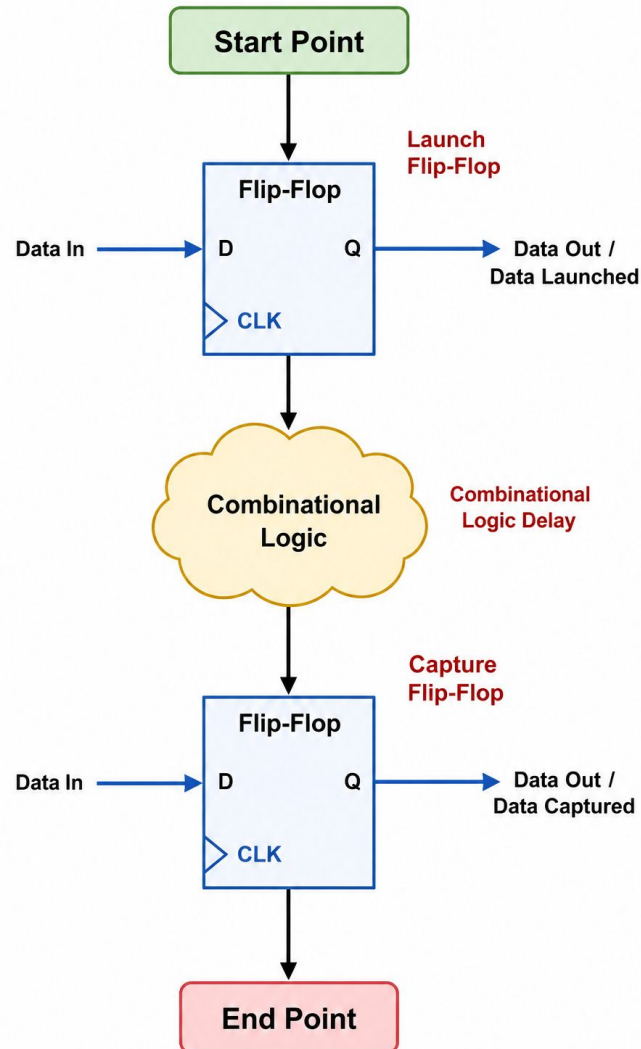
**STA determines whether data can propagate through every timing path within the available timing window.**



# Concept of STA

STA breaks a digital design into **timing paths**.

A typical path is:



STA calculates:

- Data arrival time
- Required arrival time
- Data path delay
- Clock path delay
- Setup slack
- Hold slack

If the slack is positive or zero, the path generally satisfies that timing requirement.

### **3. Timing Path**

A **timing path** is a signal path between a starting point and an ending point.

Common timing paths include:

#### **A. Register-to-Register**

FF1 → Combinational Logic → FF2

This is the most common path in synchronous digital circuits.

#### **B. Input-to-Register**

Input Port → Combinational Logic → FF

#### **C. Register-to-Output**

FF → Combinational Logic → Output Port

#### **D. Input-to-Output**

Input Port → Combinational Logic → Output Port

# Important Timing Parameters

Several timing parameters are used during STA.

## 4.1 Clock Period

The clock period is the time between two consecutive clock edges.

$$T_{clk} = \frac{1}{f_{clk}}$$

For example, for a 100 MHz clock:

$$T_{clk} = \frac{1}{100 \times 10^6} = 10ns$$

Therefore, the circuit ideally has 10 ns for data propagation between two clock edges.

## 4.2 Clock-to-Q Delay

When a flip-flop receives a clock edge, its output does not change instantaneously.

The delay between the active clock edge and the corresponding output transition is called **clock-to-Q delay** ( $T_{CQ}$ ).

## 4.3 Combinational Logic Delay

The time required for data to propagate through combinational logic is called **combinational delay** ( $T_{COMB}$ ).

## 4.4 Setup Time

**Setup time** ( $T_{SETUP}$ ) is the minimum amount of time that data must remain stable **before the active clock edge**.

## 4.5 Hold Time

**Hold time** ( $T_{HOLD}$ ) is the minimum amount of time that data must remain stable **after the active clock edge**.

## 4.6 Slack

**Slack** indicates how much timing margin a path has.

### Setup Slack

Slack setup = Required Time – Arrival Time

| Slack    | Meaning                   |
|----------|---------------------------|
| Positive | Timing passes             |
| Zero     | Exactly meets requirement |
| Negative | Timing violation          |

For example:

Required Time = 10 ns

Arrival Time = 8 ns

Slack = 10 - 8

= +2 ns

The circuit has **2 ns timing margin**.

If:

Required Time = 10 ns

Arrival Time = 11 ns

Slack = -1 ns

there is a **1 ns setup violation**.

## **Clock Skew**

**Clock skew** is the difference in arrival time of the clock signal at two sequential elements.

There are two basic types:

- Positive skew
- Negative skew

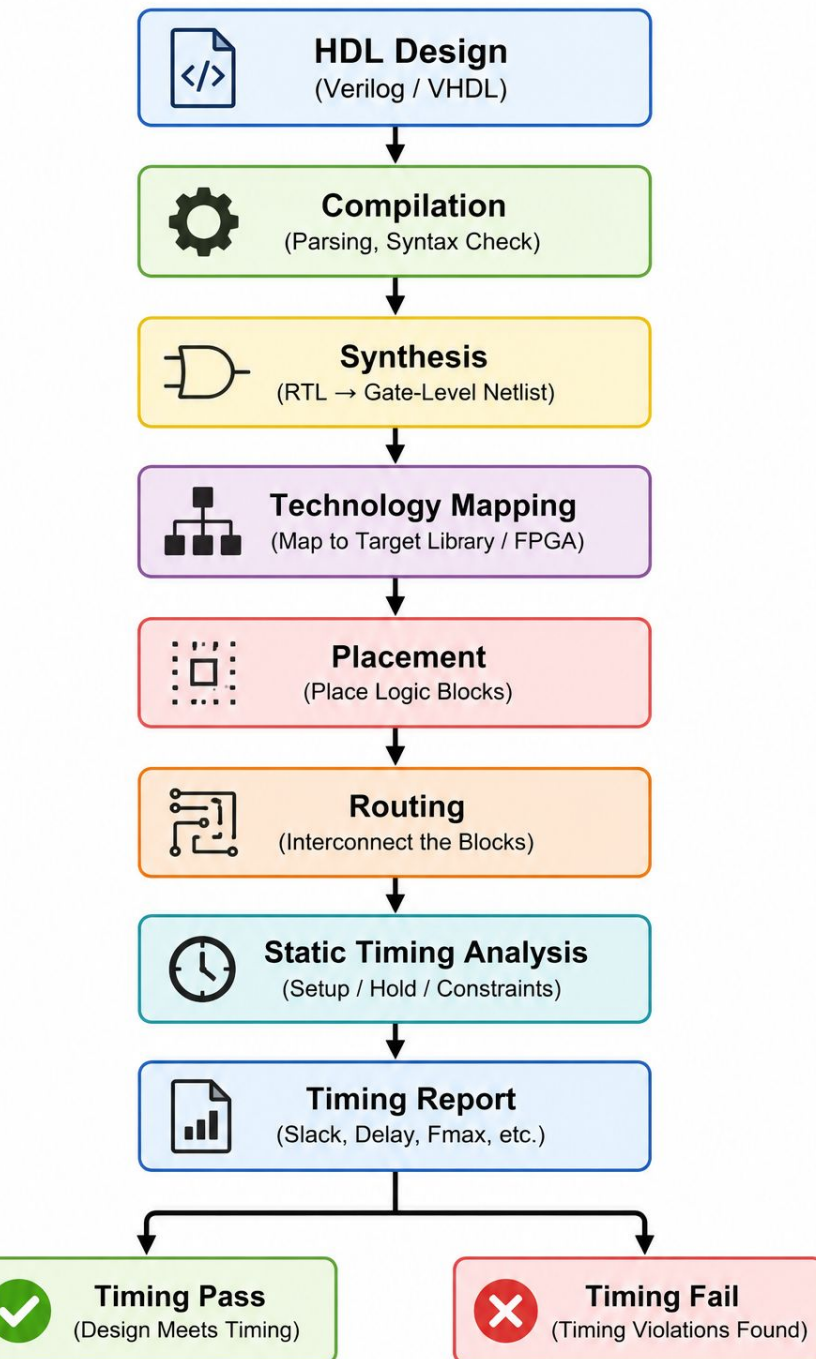
Clock skew can affect both setup and hold timing.

Therefore, STA analyzes the actual clock distribution network rather than assuming that every flip-flop receives the clock simultaneously.

## **Clock Jitter**

**Clock jitter** refers to unwanted variation in the timing of clock edges.

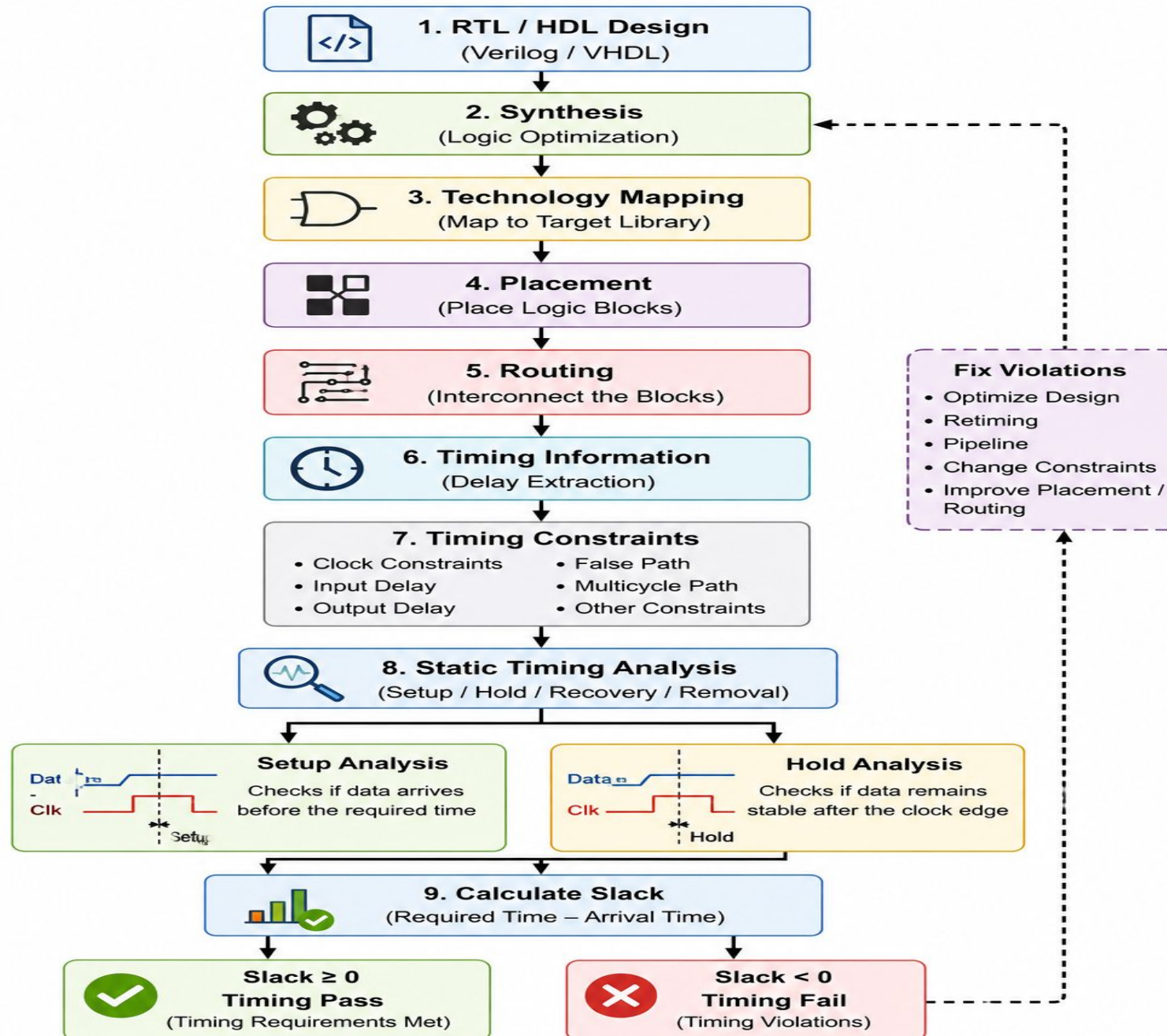
# STA in FPGA Design



STA uses the actual implementation information such as:

- LUT delays
- Flip-flop delays
- Routing delays
- Clock network delays
- I/O delays
- Device-specific timing characteristics

# Complete STA Flow





# One-Hot Encoding

**One-hot encoding** is a technique used in **digital system design**, **FPGA design**, and **finite state machine (FSM) implementation** where **each state is represented by a separate flip-flop or binary bit**.

For an FSM with N states, one-hot encoding generally uses **N flip-flops**, unlike binary encoding, which requires approximately:  $\lceil \log_2 N \rceil$  flip-flops.

The main advantage of one-hot encoding is that **only one bit is HIGH (1) at any time**, while all other state bits remain LOW (0).

## 1. Basic Concept

Suppose an FSM has four states:

S0, S1, S2, S3

In one-hot encoding, they are represented as:

| State | Q3 | Q2 | Q1 | Q0 |
|-------|----|----|----|----|
| S0    | 0  | 0  | 0  | 1  |
| S1    | 0  | 0  | 1  | 0  |
| S2    | 0  | 1  | 0  | 0  |
| S3    | 1  | 0  | 0  | 0  |

Notice that **exactly one bit is 1** for every valid state.

Hence the name **one-hot**:

**One state bit is "hot" (logic 1) at a time.**

# One-Hot vs Binary Encoding

Consider an FSM containing **8 states**.

## Binary encoding

Number of flip-flops required:

$$\lceil \log_2 8 \rceil = 3$$

States can be represented as:

| State | Binary |
|-------|--------|
| S0    | 000    |
| S1    | 001    |
| S2    | 010    |
| S3    | 011    |
| S4    | 100    |
| S5    | 101    |
| S6    | 110    |
| S7    | 111    |

## One-hot encoding

Eight flip-flops are required:

| State | One-Hot  |
|-------|----------|
| S0    | 00000001 |
| S1    | 00000010 |
| S2    | 00000100 |
| S3    | 00001000 |
| S4    | 00010000 |
| S5    | 00100000 |
| S6    | 01000000 |
| S7    | 10000000 |

# General One-Hot Encoding

For an FSM having N states:

Flip-Flops required=N

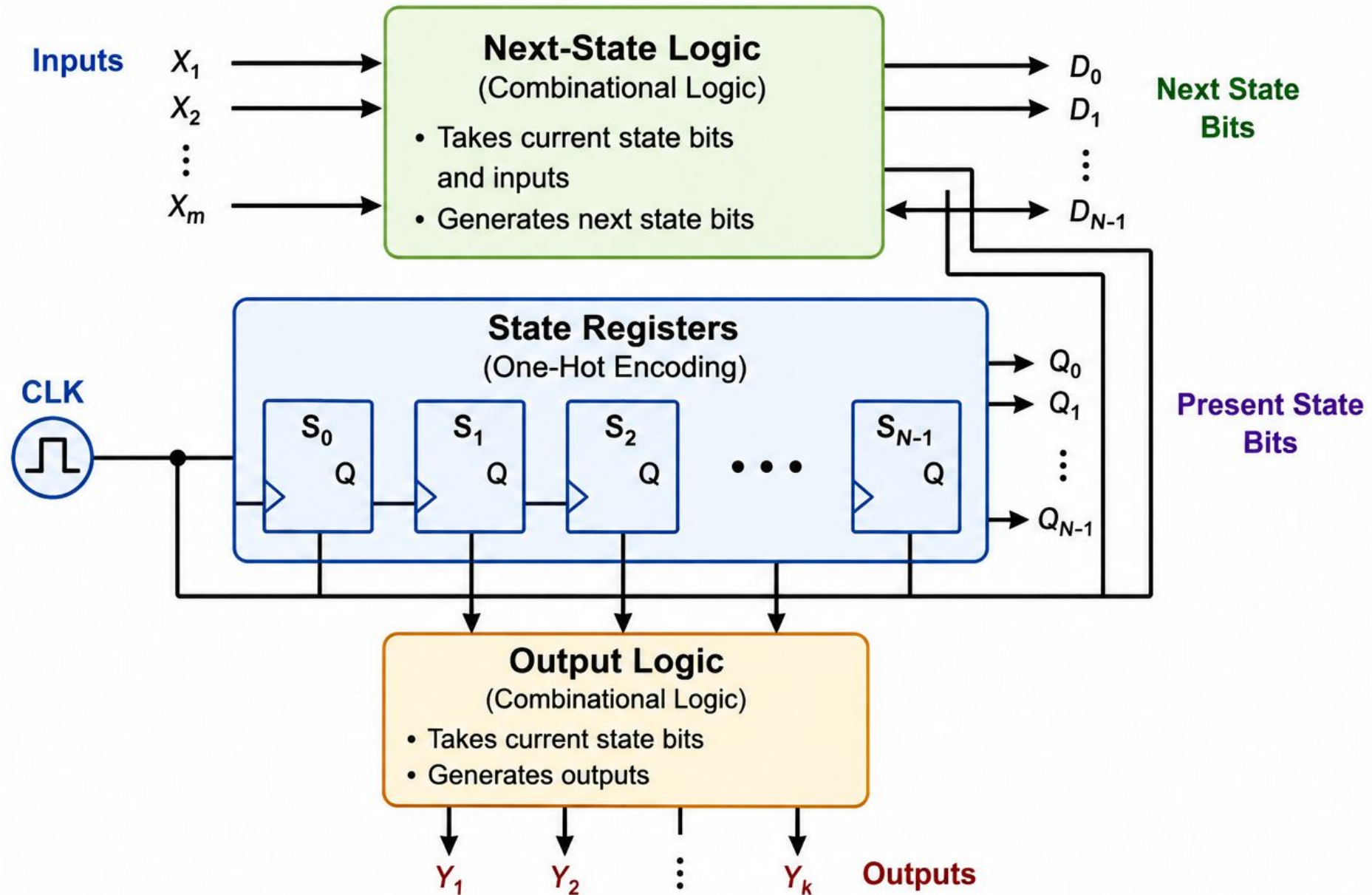
For example:

| Number of States | Binary FFs | One-Hot FFs |
|------------------|------------|-------------|
| 2                | 1          | 2           |
| 4                | 2          | 4           |
| 8                | 3          | 8           |
| 16               | 4          | 16          |
| 32               | 5          | 32          |
| 64               | 6          | 64          |

Thus, one-hot encoding requires significantly more flip-flops.

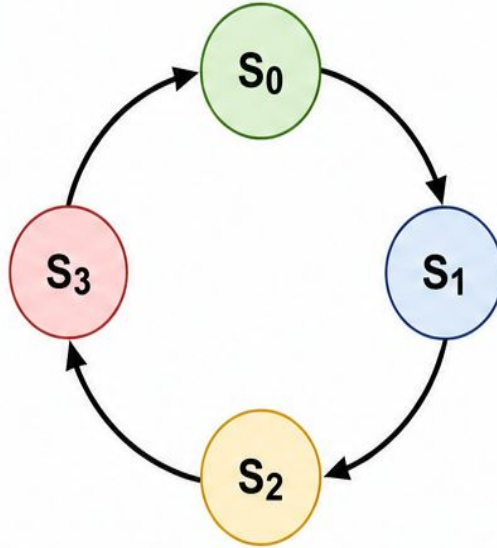
However, in an **FPGA**, flip-flops are usually abundant, so the additional flip-flops may be an acceptable trade-off for simpler and faster logic.

# One-Hot FSM Architecture



# Example: Four-State FSM

## 1. Four-State FSM (State Diagram)

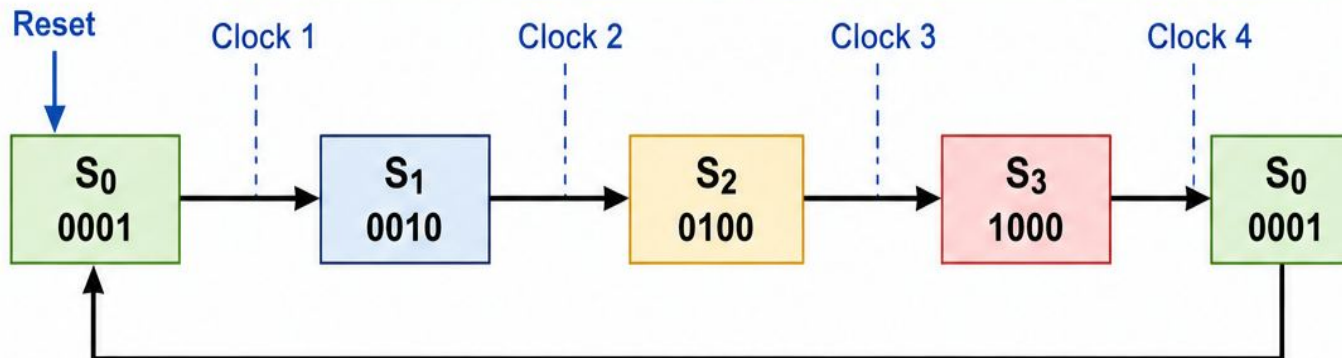


## 2. One-Hot Encoding

| State          | Q <sub>3</sub> | Q <sub>2</sub> | Q <sub>1</sub> | Q <sub>0</sub> |
|----------------|----------------|----------------|----------------|----------------|
| S <sub>0</sub> | 0              | 0              | 0              | 1              |
| S <sub>1</sub> | 0              | 0              | 1              | 0              |
| S <sub>2</sub> | 0              | 1              | 0              | 0              |
| S <sub>3</sub> | 1              | 0              | 0              | 0              |

Only one bit is '1' at any valid state.

## 3. State Transition Sequence (One-Hot Representation)



### Explanation

- FSM starts in S<sub>0</sub> = 0001
- After each clock, it moves to the next state
- At any time, only one state bit is HIGH (1)

This is One-Hot Encoding: For N states, N flip-flops are used.

# Why One-Hot Encoding is Fast

The major advantage comes from **simplified next-state logic**.

Suppose the FSM is currently in state S2:

$$S2 = 0100$$

The state decoder does not need to decode multiple binary bits to determine the current state.

The corresponding state bit itself indicates the state:

$$S2=1$$

Therefore, the next-state logic can often be implemented using relatively simple logic equations.

For example:

$$D_{S3} = S2$$

means:

If the FSM is currently in S2, the next state is S3.

This can reduce logic depth.

# Example of Next-State Equations

Consider a four-state FSM:

$S0 \rightarrow S1 \rightarrow S2 \rightarrow S3 \rightarrow S0$

The state equations can be written as:

$D0=S3 \ D1=S0 \ D2=S1 \ D3=S2$

The hardware therefore becomes approximately:

$S3 \longrightarrow \blacktriangleright D0$

$S0 \longrightarrow \blacktriangleright D1$

$S1 \longrightarrow \blacktriangleright D2$

$S2 \longrightarrow \blacktriangleright D3$

This is extremely simple compared with decoding a multi-bit binary state.

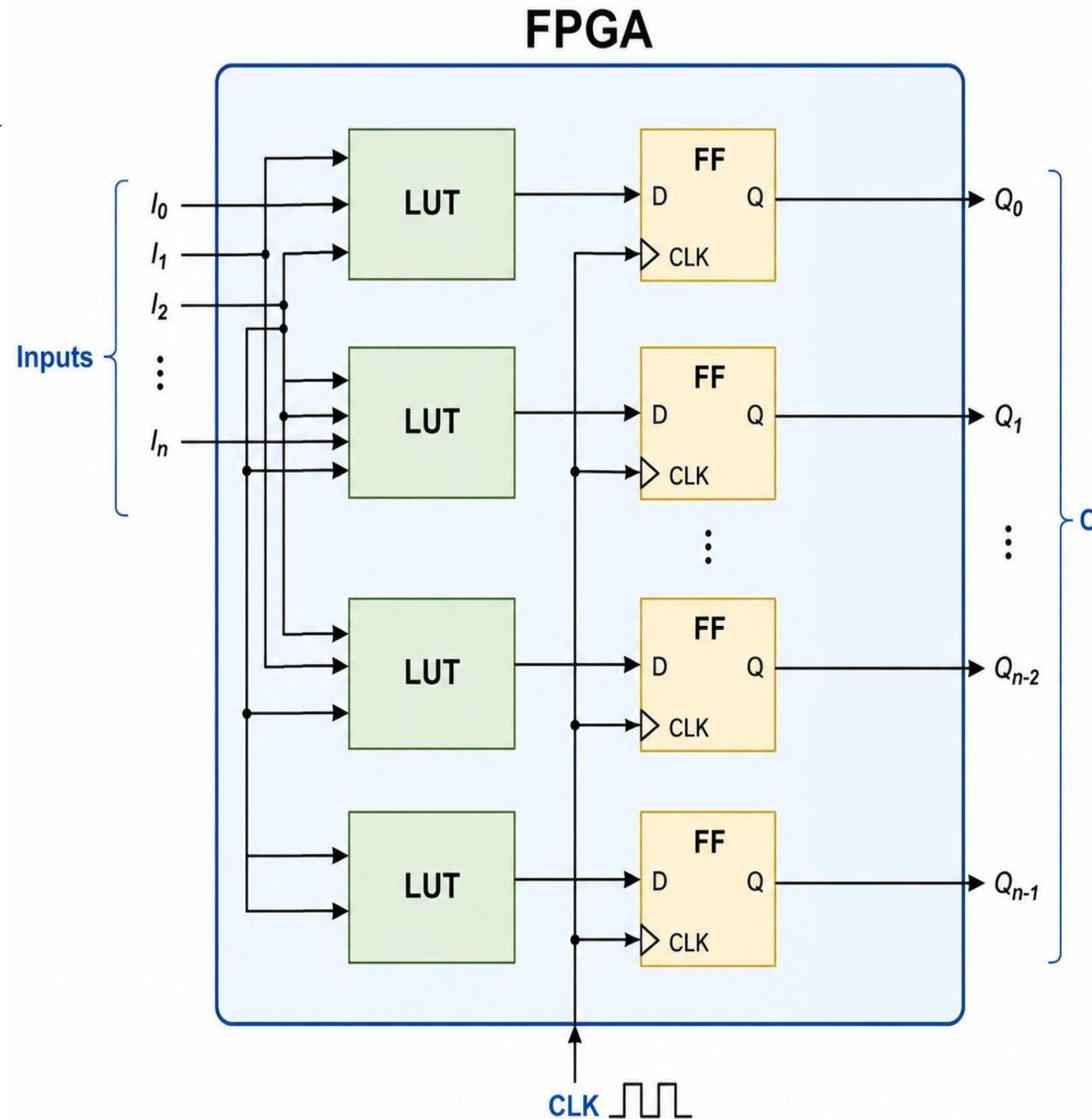
# One-Hot Encoding in FPGA

One-hot encoding is particularly popular in **FPGA-based FSM implementations**.

An FPGA contains many configurable resources, including:

- LUTs
- Flip-flops
- Routing resources
- Block RAM
- DSP blocks

Since FPGAs generally provide a large number of flip-flops, using one flip-flop per state can be practical.





# Advantages of One-Hot Encoding

## **1. Simple decoding**

Since one state bit represents each state, state decoding is straightforward.

## **2. High speed**

Reduced combinational logic can result in shorter critical paths.

## **3. Simple next-state logic**

Many state transitions can be implemented using simple Boolean expressions.

## **4. FPGA-friendly**

FPGAs typically have many flip-flops, making the extra registers less costly than they might be in an ASIC.

## **5. Easier state observation**

During debugging, the active state can often be identified directly from the state bits.

## **6. Reduced logic depth**

The next-state logic can be simpler, potentially improving timing performance.

# Disadvantages of One-Hot Encoding

## 1. More flip-flops

For N states:

N FFs

are required.

## 2. Increased area in some technologies

In ASICs, using one flip-flop per state can consume significantly more silicon area.

## 3. More power

More flip-flops and associated clock loading can increase clock power.

## 4. Invalid states

For a 4-state FSM, valid states are:

0001

0010

0100

1000

But combinations such as:

0000

0011

0110

1111

are invalid.

The FSM should therefore include appropriate recovery/default logic.

# When Should One-Hot Encoding Be Used?

One-hot encoding is particularly useful when:

- The FSM has a **small or moderate number of states**.
- The target is an **FPGA**.
- High operating frequency is important.
- Simple next-state logic is desirable.
- The FPGA has sufficient flip-flops.
- FSM timing is more important than minimizing register count.

For a very large FSM, one-hot encoding may consume many flip-flops and may not be the best choice.

# Xilinx Virtex-II Pro FPGA

- **FPGA family** based on the **Virtex-II architecture**.
- Combines **programmable FPGA logic** with embedded processing.
- The important features of Virtex-II Pro include:
- Embedded **IBM PowerPC 405 (PPC405) RISC processor**
- **RocketIO multi-gigabit transceivers**
- Configurable Logic Blocks (CLBs)
- Block SelectRAM
- Embedded multipliers
- SelectIO-Ultra I/O resources
- Digital Clock Managers (DCMs)
- Programmable interconnect/routing
- Dedicated processor-to-FPGA interfaces.

**Virtex-II Pro = Virtex-II FPGA Fabric + Embedded PowerPC 405 + RocketIO**

## 6. Block SelectRAM

Virtex-II Pro contains dedicated on-chip memory resources called **Block SelectRAM**.

They can be used for:

- FIFOs
- Buffers
- Lookup tables
- Processor memory
- Data storage
- Pipeline buffering

Using dedicated Block RAM is generally much more efficient than constructing large memories from ordinary LUT resources.

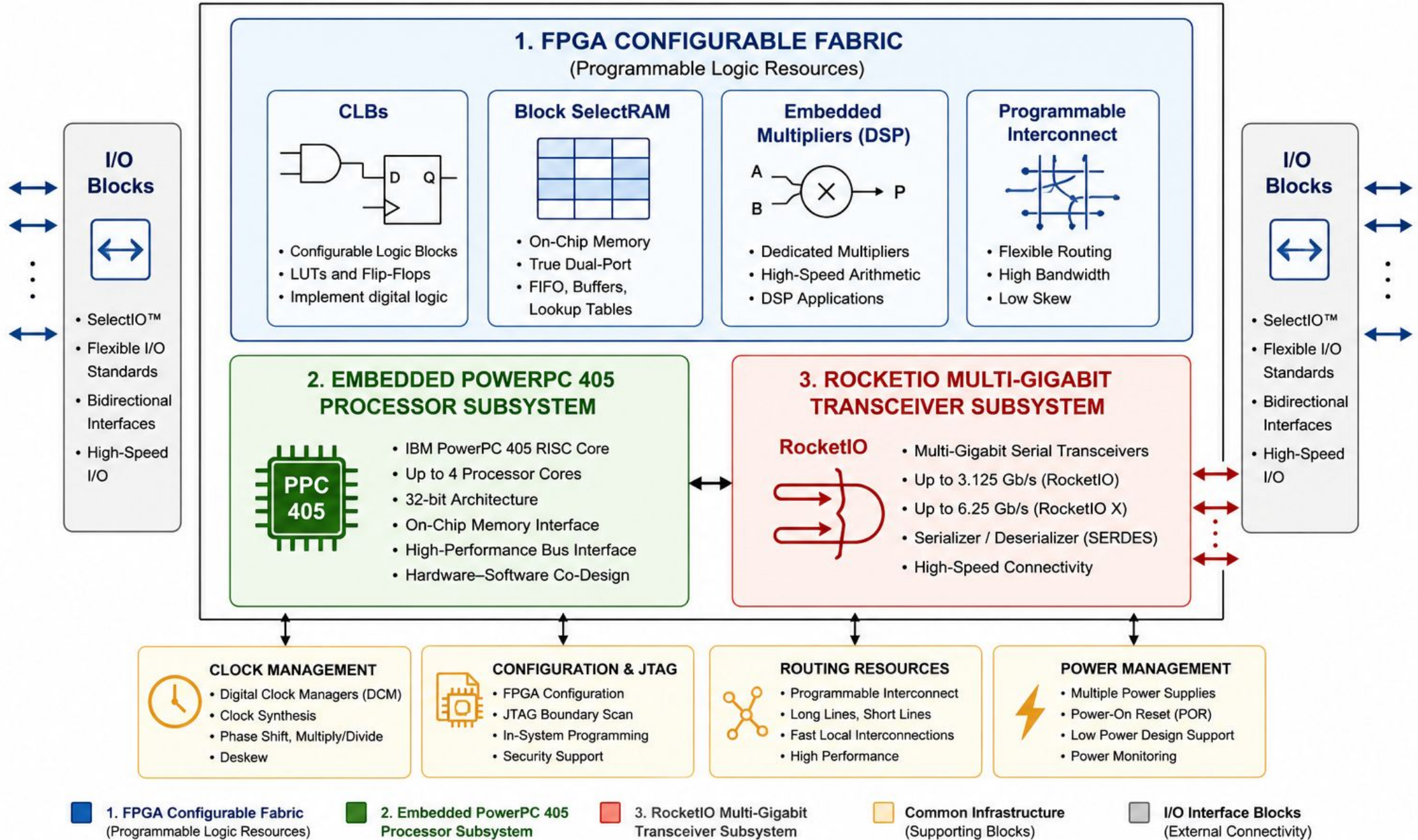
## 11. RocketIO Multi-Gigabit Transceivers

Another major feature is **RocketIO**, Xilinx's high-speed serial transceiver technology.

The documented Virtex-II Pro devices supported RocketIO transceivers up to approximately **3.125 Gb/s**; the later Virtex-II Pro X family used **RocketIO X**, supporting up to approximately **6.25 Gb/s**.

# Virtex-II Pro – Basic Architecture

## Three Major Subsystems



# Embedded System on Programmable Chip

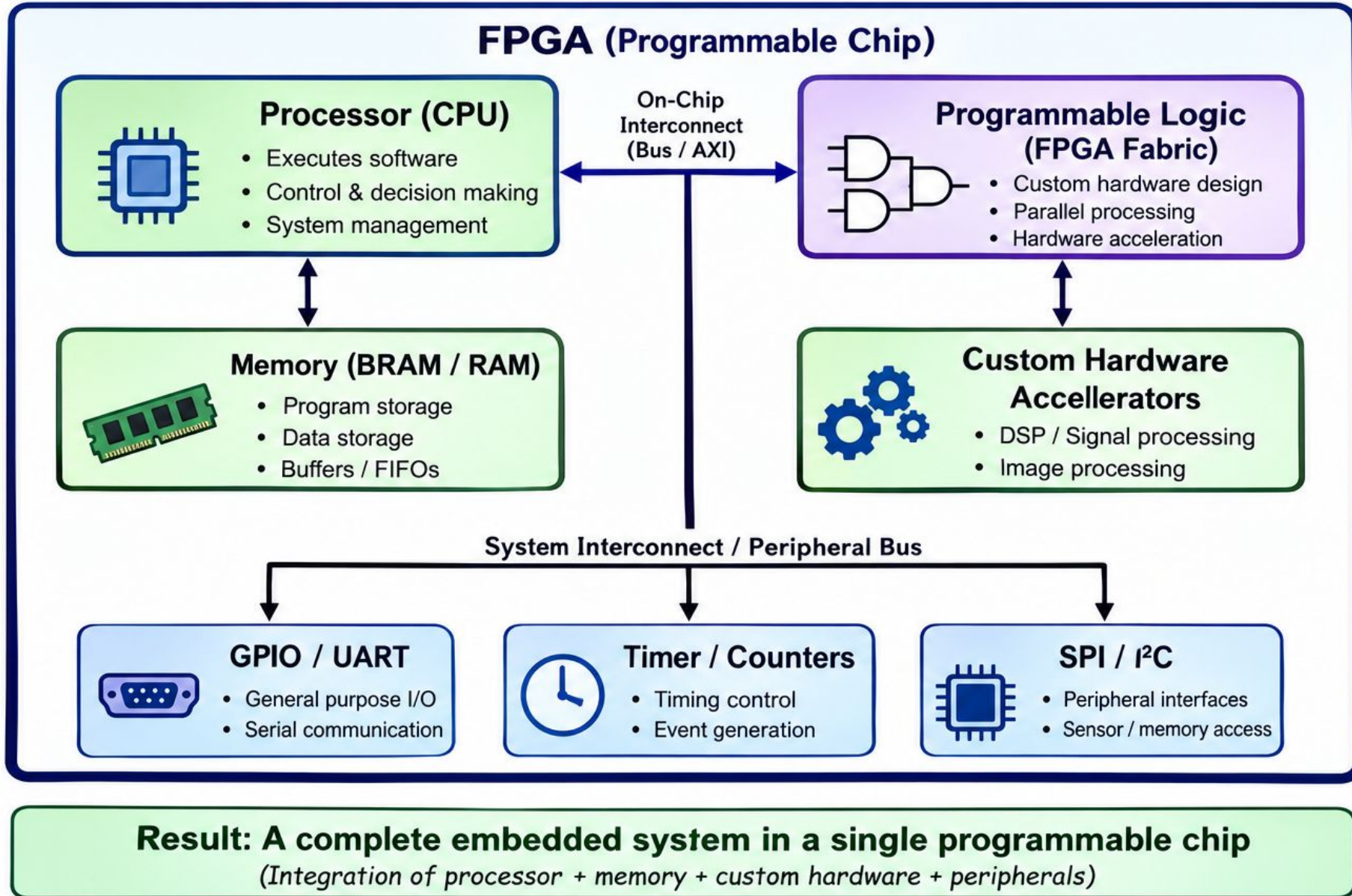
**Embedded System on Programmable Chip (SoPC)** is a system-design approach in which a complete embedded system is implemented on a **programmable device such as an FPGA**. Instead of using separate chips for the processor, memory, peripherals, and custom hardware, many of these components can be integrated into a single programmable device.

A simple representation is:



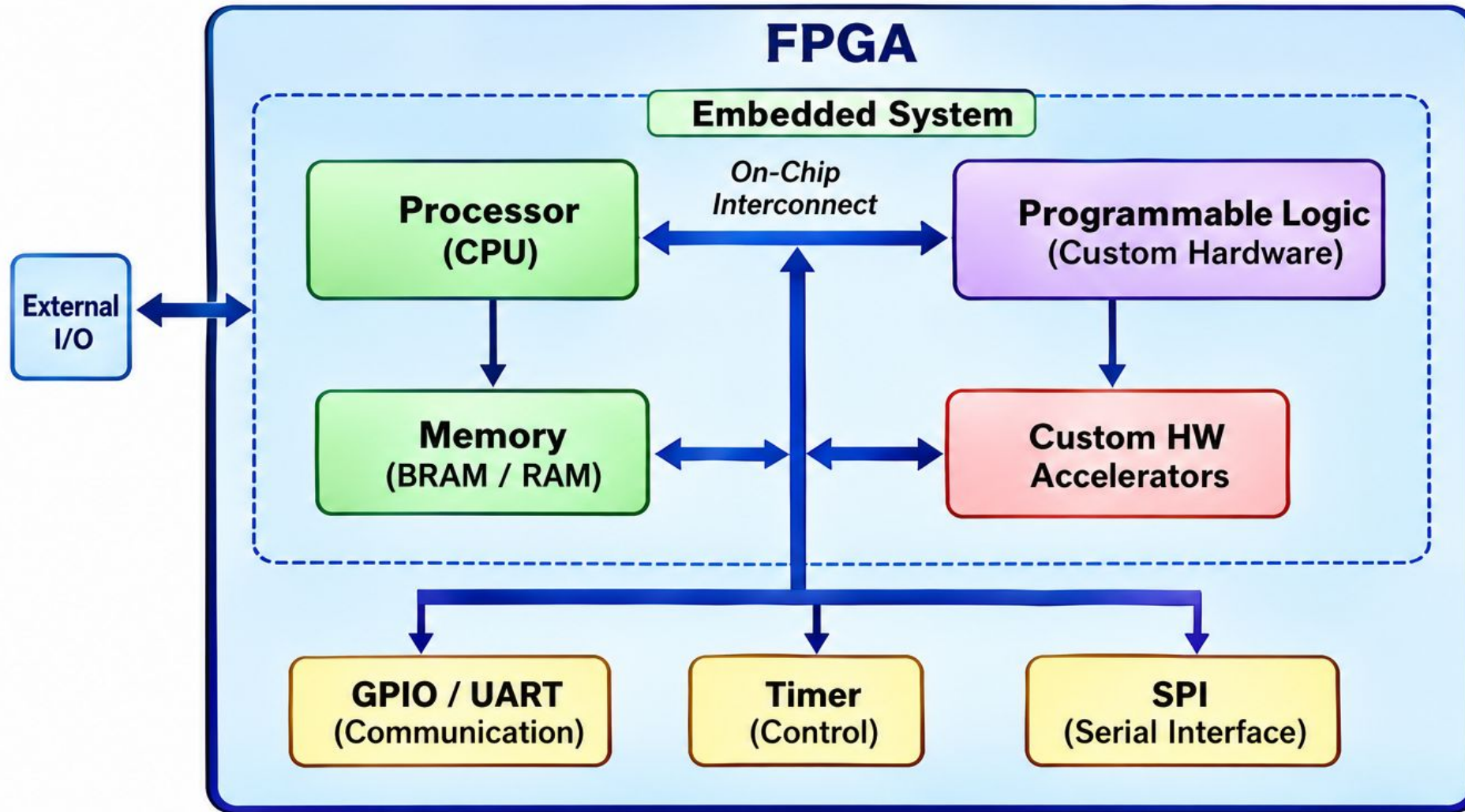
# Embedded System on Programmable Chip

(SoP/C / FPGA-based Embedded System)





# Embedded System on Programmable Chip (SoPC)



## Key Features:

- ✓ Processor + Programmable Logic on a Single Chip
- ✓ Custom Hardware Accelerators
- ✓ Hardware/Software Co-Design
- ✓ Reconfigurable and Flexible System

# Major Components of an SoPC

SoPC = Processor + Memory + Peripherals + Custom FPGA Hardware + Software

## **Hardware–Software Co-Design**

### **Software is suitable for:**

- Control
- Decision making
- Configuration
- Complex algorithms
- User interfaces

### **Hardware is suitable for:**

- High-speed operations
- Parallel processing
- Repetitive calculations
- Real-time processing
- Hardware acceleration

# Advantages of Hardware/Software Partitioning

| Task                     | Implementation       |
|--------------------------|----------------------|
| System control           | Software             |
| Image filtering          | Hardware             |
| Communication management | Software             |
| Encryption               | Hardware accelerator |
| Sensor interface         | Hardware             |

This allows each portion of the system to be implemented using the most appropriate technology.

# SoPC vs Conventional Embedded System

| Feature                     | Conventional Embedded System | SoPC                           |
|-----------------------------|------------------------------|--------------------------------|
| Processor                   | Separate IC                  | Integrated/implemented in FPGA |
| Memory                      | External IC                  | On-chip + external             |
| Custom hardware             | Separate IC                  | FPGA fabric                    |
| Peripherals                 | Separate IC/IP               | FPGA IP                        |
| Hardware acceleration       | Limited                      | Excellent                      |
| Parallel processing         | Limited                      | High                           |
| Reconfigurability           | Low                          | High                           |
| Integration                 | Lower                        | High                           |
| Hardware/software co-design | Limited                      | Excellent                      |

# Hardware–Software Co-Simulation

- Hardware–software co-simulation is a design and verification technique in which the hardware portion and software portion of an embedded system are developed, executed, and tested together in a coordinated simulation environment.
- It is especially important in SoC, FPGA-based embedded systems, and hardware–software co-design, where some functions are implemented in hardware while others run as software on a processor.

## 1. Basic Concept

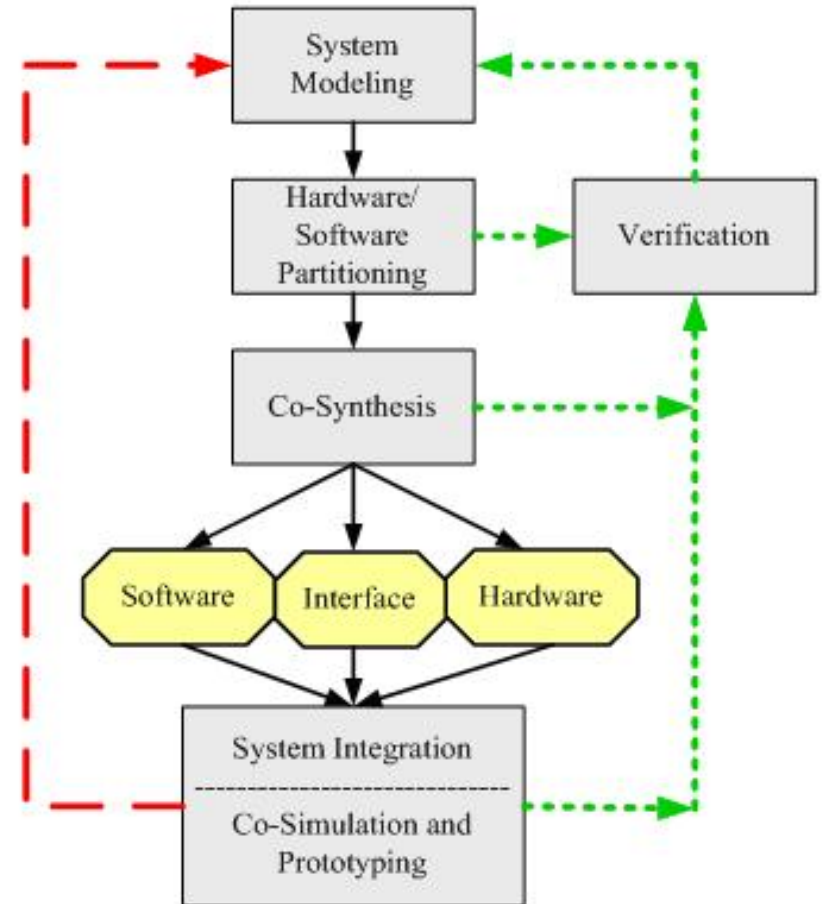
A typical embedded system contains:

**Hardware:** FPGA logic, ASIC blocks, peripherals, memory controllers, accelerators, etc.

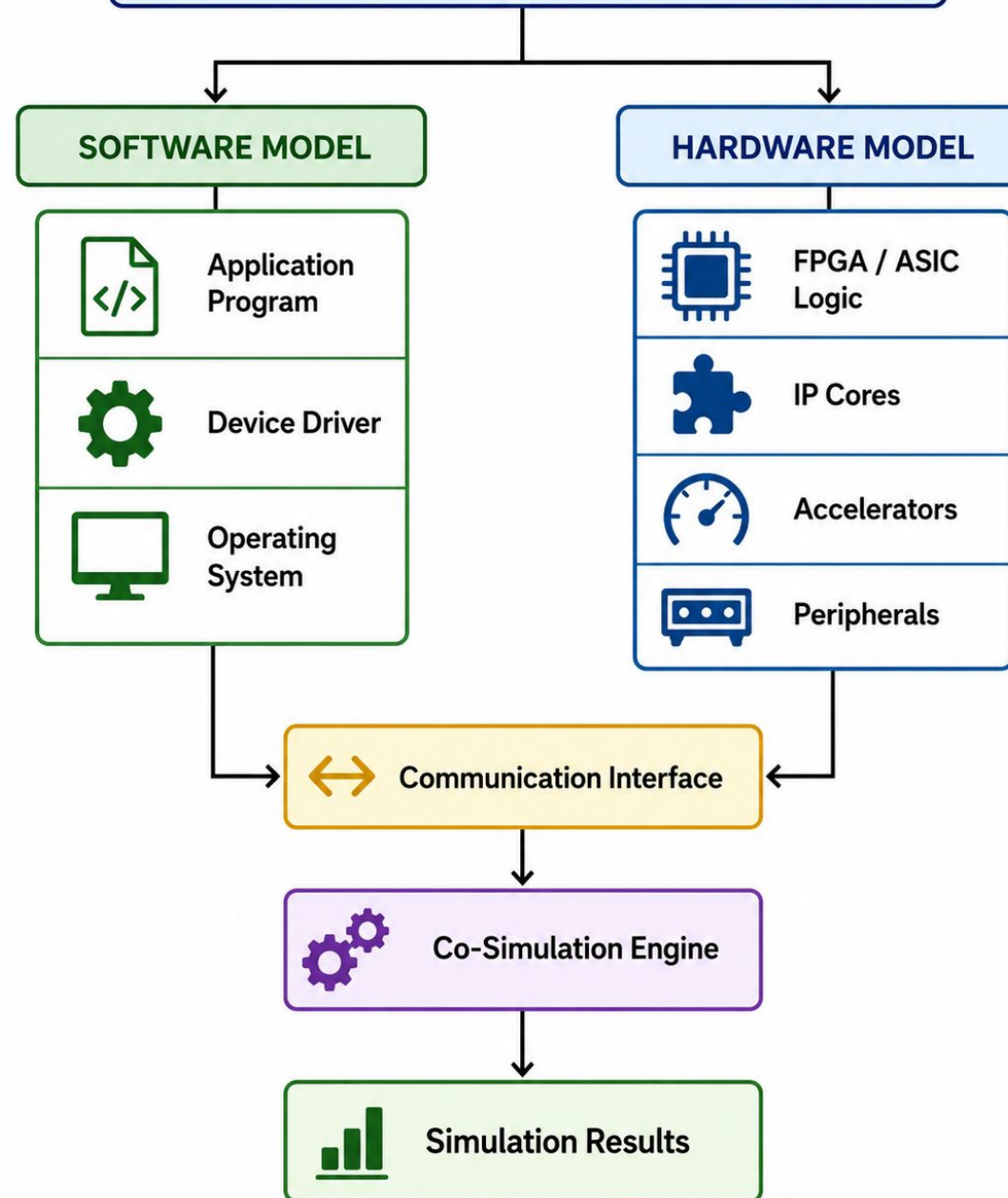
**Processor:** Microprocessor, Microcontroller, or soft/hard processor core.

**Software:** Firmware, device drivers, application programs, operating system, etc.

In co-simulation, these components are simulated together so that the designer can verify how they interact before implementing the complete system on physical hardware.

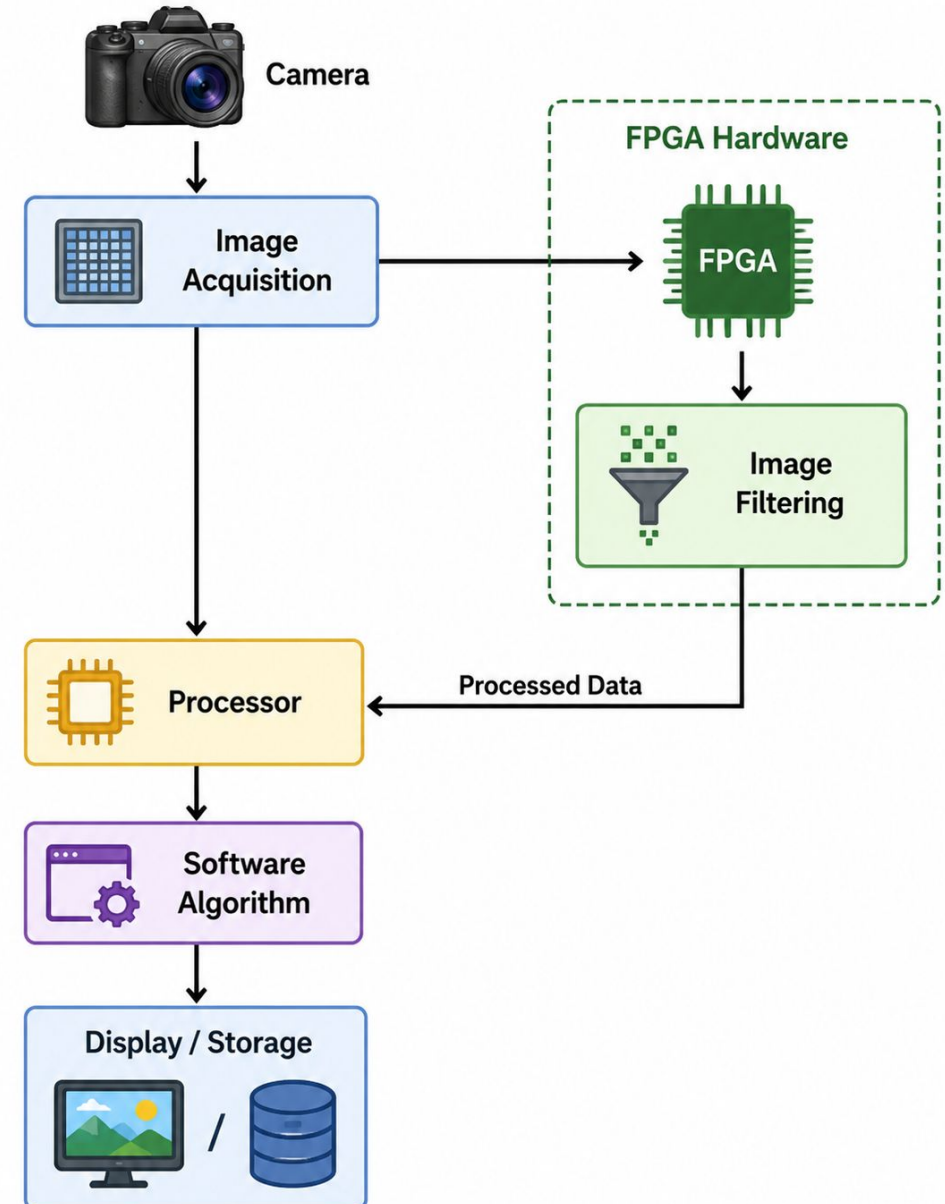


# Hardware-Software Co-Simulation



# Why Hardware–Software Co-Simulation is Needed

- Modern embedded systems are not purely hardware or software systems.
- A designer may implement computationally intensive image filtering in FPGA hardware while implementing decision-making and user-interface functions in software.
- Co-simulation allows both portions to be tested together.



# Main Objectives

Hardware–software co-simulation is used to:

## **1. Verify hardware–software interaction**

It checks whether the processor software correctly communicates with hardware peripherals and accelerators.

## **2. Detect interface problems**

For example:

- Incorrect data width
- Incorrect address mapping
- Incorrect control signals
- Incorrect timing
- Communication protocol errors

## **3. Evaluate system performance**

The designer can measure:

- Execution time
- Hardware latency
- Software execution time
- Throughput
- Communication overhead

## **4. Explore hardware/software partitioning**

A function can initially be implemented in software and then moved to hardware to determine whether performance improves.

## **5. Reduce development time**

Errors can be detected in simulation before fabricating an ASIC or programming/deploying the final FPGA system.



# Hardware–Software Partitioning

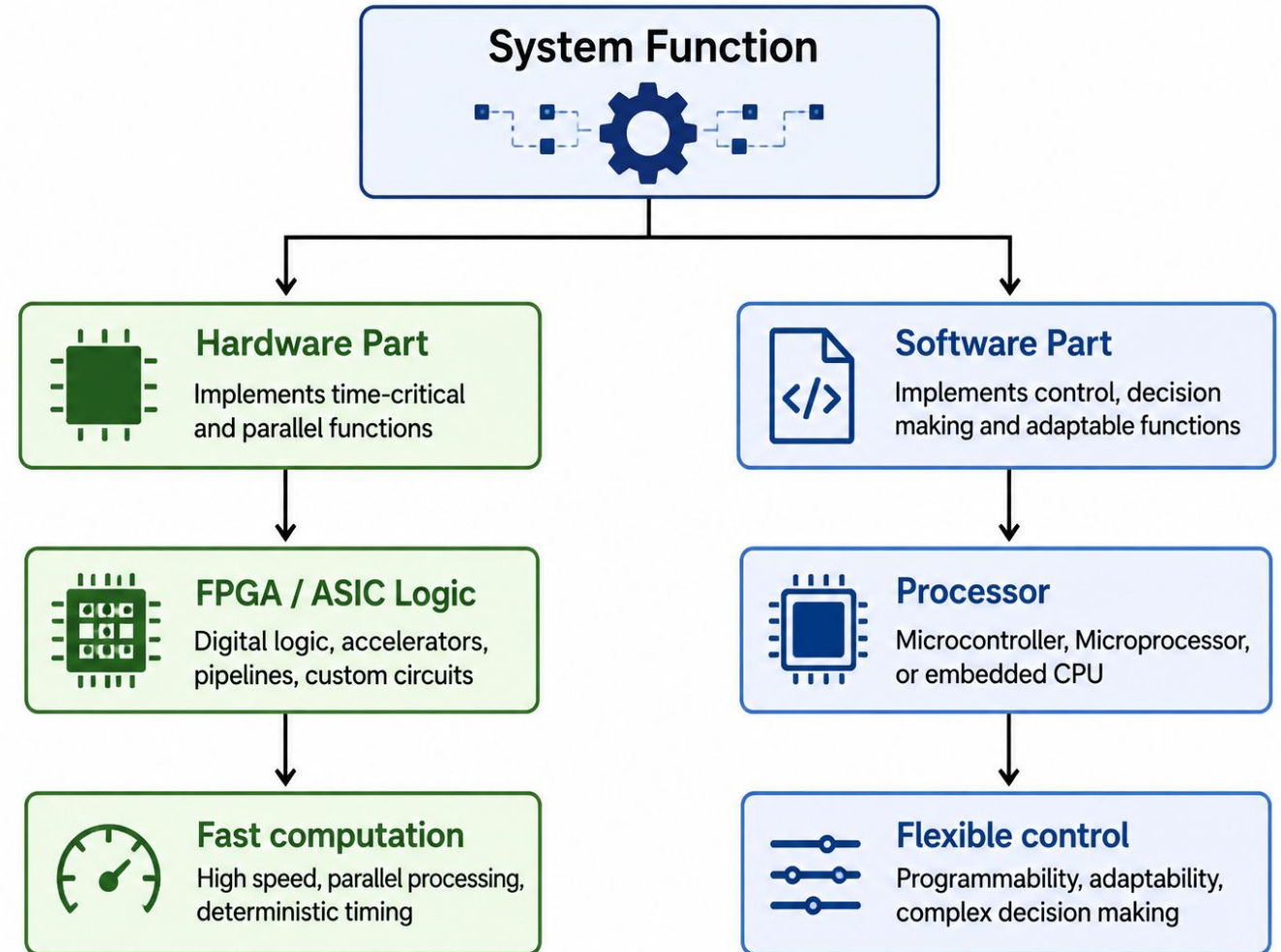
A system function is divided into two categories:

## Hardware is preferred when:

- High speed is required
- Parallel processing is required
- Computation is repetitive
- Deterministic timing is important
- Hardware acceleration is beneficial

## Software is preferred when:

- Algorithm changes frequently
- Flexibility is important
- Complex control operations are required
- Development time should be reduced
- The computation is not performance-critical



# Co-Simulation vs Traditional Simulation

| Feature                       | Traditional Hardware Simulation | Hardware–Software Co-Simulation |
|-------------------------------|---------------------------------|---------------------------------|
| Hardware verification         | Yes                             | Yes                             |
| Software verification         | Limited                         | Yes                             |
| Processor model               | Usually absent                  | Present                         |
| Hardware/software interaction | Limited                         | Central focus                   |
| Interface verification        | Limited                         | Extensive                       |
| Performance analysis          | Hardware-focused                | System-level                    |
| System partitioning           | Difficult                       | Can be evaluated                |
| Embedded-system verification  | Limited                         | Highly suitable                 |

# Advantages

## **Functional verification**

Hardware and software can be tested together before implementation.

## **Early error detection**

Interface and integration errors can be identified early.

## **Reduced hardware cost**

A physical prototype is not immediately required.

## **Faster design exploration**

Different hardware/software partitioning strategies can be evaluated.

## **Performance estimation**

The designer can determine whether a function should remain in software or be accelerated in hardware.

## **Improved reliability**

More system-level test cases can be performed before deployment.

## **Parallel development**

Hardware and software teams can develop and test their components using a common system model.

# Applications

Hardware–software co-simulation is widely used in:

- **System-on-Chip (SoC) design**
- **System-on-Programmable-Chip (SoPC)**
- FPGA-based embedded systems
- Automotive electronics
- Robotics
- Digital signal processing
- Image and video processing
- Wireless communication
- IoT devices
- AI/ML hardware accelerators
- Consumer electronics
- Aerospace and defense electronics

# Hardware–Software Co-Simulation vs Co-Design

These terms are related but different.

**Hardware–software co-design** focuses primarily on **designing and partitioning** a system between hardware and software.

**Hardware–software co-simulation** focuses on **simulating and verifying the interaction** between those hardware and software components.

In simple terms:

**Co-design = Decide and build what goes into hardware and software.**

**Co-simulation = Test hardware and software working together.**

# Debugging FPGA Design

**FPGA design debugging** is the systematic process of identifying, locating, and correcting errors in an FPGA-based digital system. It involves checking the design at different stages—from HDL coding and simulation to synthesis, implementation, timing analysis, FPGA programming, and real-hardware testing.

The main objective is to ensure that:

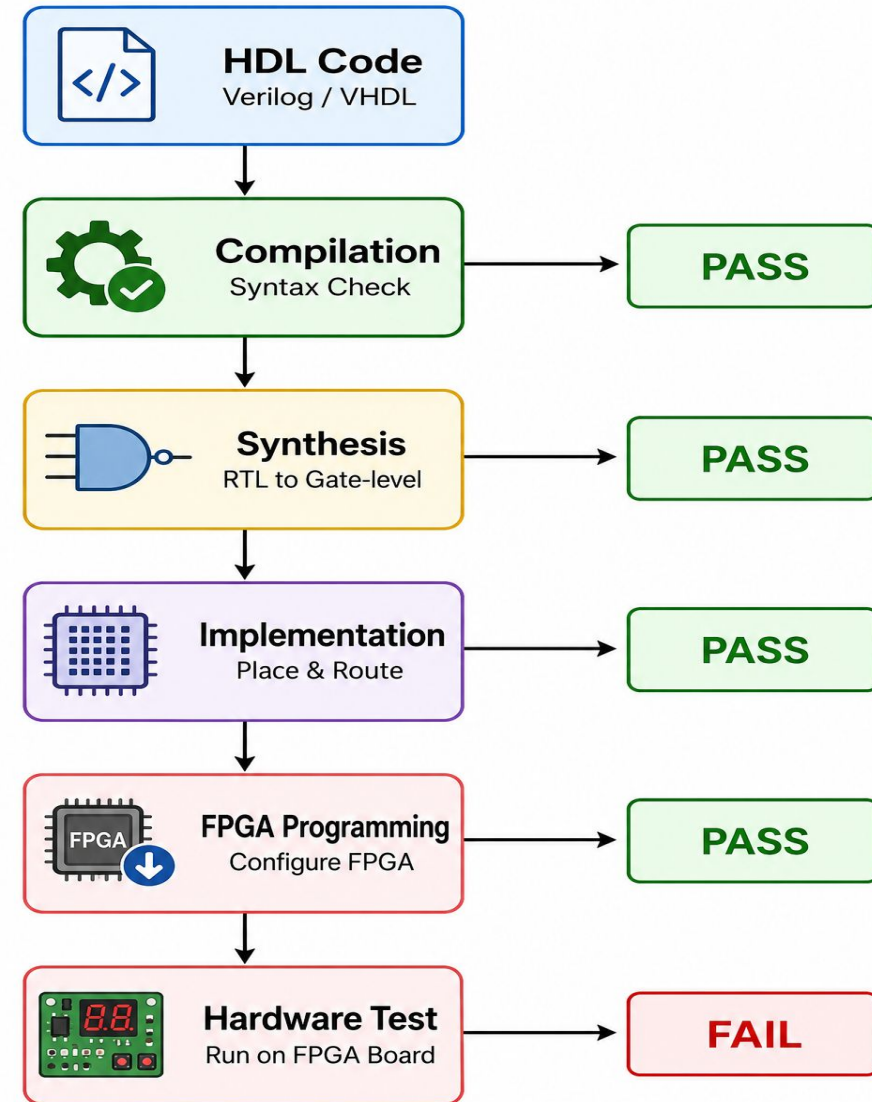
The actual FPGA hardware behaves exactly as intended by the design specification.

# Why FPGA Debugging is Required

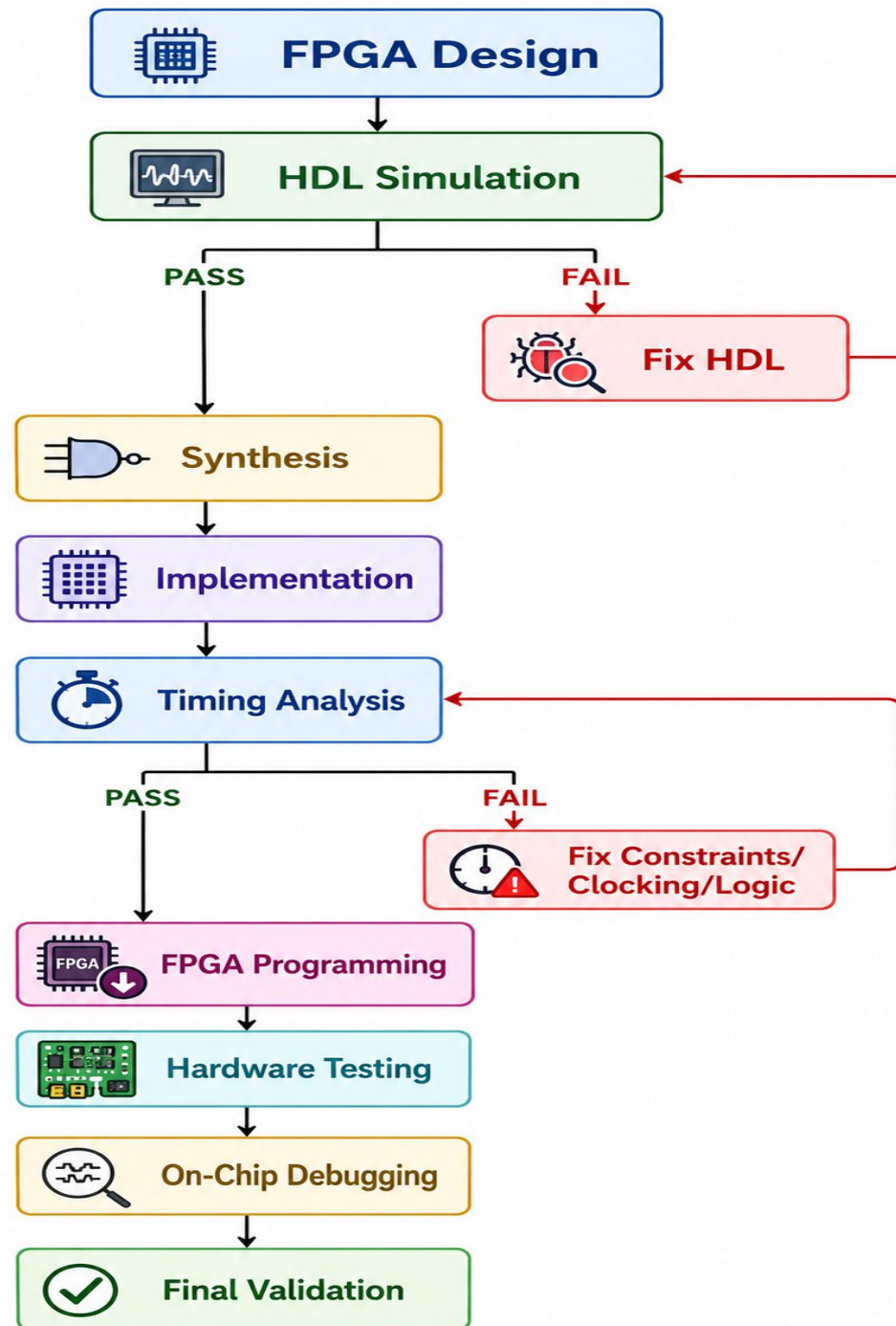
An FPGA design can contain several types of errors:

- HDL syntax errors
- Incorrect logic/functionality
- Incorrect state-machine transitions
- Clocking problems
- Reset problems
- Setup/hold violations
- Incorrect pin assignments
- Signal-integrity problems
- Incorrect constraints
- Hardware/software interface errors
- Incorrect initialization
- Metastability
- Unexpected synthesis behavior

A design may therefore **compile successfully but still fail on the actual FPGA board.**



## FPGA Debugging Flow





# Levels of FPGA Debugging

FPGA debugging can be divided into several levels.

## Level 1 — HDL Debugging

Check:

- Syntax
- Data types
- Operators
- Signal assignments
- Processes
- Clock/reset logic

## Level 2 — Functional Simulation

Verify whether the RTL behaves correctly for different input conditions.

## Level 3 — Synthesis Debugging

Check whether synthesis has correctly interpreted the HDL.

## Level 4 — Timing Debugging

Verify:

- Setup time , Hold time , Clock frequency , Clock skew , Critical paths

## Level 5 — Implementation Debugging

Check:

- Placement , Routing , Resource utilization , I/O assignments  
Timing constraints

## Level 6 — Hardware Debugging

Test the programmed FPGA using:

- LEDs
- Switches
- UART
- Logic analyzer
- Oscilloscope
- On-chip logic analyzer